

Seralix: A Minimal Language Prototype for Expressing Fast Fourier Transform Algorithms

David Mateo Barbosa Monsalve
Universidad de los Andes
Systems and Computing Engineering

Abstract—This paper presents the implementation of the Fast Fourier Transform (FFT) as the central objective of the project. To evaluate the feasibility of implementing this algorithm from first principles, a minimal programming language named Seralix was designed and used as an execution environment. The results demonstrate that FFT can be successfully implemented within a constrained computational system while preserving its theoretical complexity.

I. INTRODUCTION

The Fast Fourier Transform (FFT) is a fundamental algorithm in computer science, enabling efficient computation of the Discrete Fourier Transform (DFT) in $O(n \log n)$ time. It plays a critical role in areas such as signal processing, numerical analysis, and polynomial multiplication.

Unlike its naive counterpart, which operates in quadratic time, FFT achieves its efficiency through recursive decomposition and structured data manipulation. However, implementing FFT correctly requires a precise understanding of recursion, indexing patterns, and numerical operations over the complex domain.

In most programming environments, these mechanisms are abstracted away by high-level constructs and optimized libraries. As a result, the underlying computational process is often treated as a black box.

To address this, the present project adopts a different approach: instead of relying on existing languages, a minimal execution environment was constructed to explicitly control and observe each stage of computation. This environment, implemented as a simple programming language named Seralix, is not the objective of the project, but rather a tool used to validate the implementation process.

The central objective of this work is therefore to demonstrate that the Fast Fourier Transform can be correctly implemented from first principles within a minimal computational system, preserving its theoretical complexity and operational structure.

II. PROBLEM STATEMENT

Implementing FFT requires several computational capabilities:

- **Recursive decomposition:** The algorithm splits a sequence into even and odd indexed elements.

```
even = fft(a[0::2])  
odd = fft(a[1::2])
```

- **Arithmetic over complex numbers:** FFT involves operations such as multiplication by complex roots of unity.

```
w *= wn
```

- **Sequence manipulation:** Efficient indexing and recombination of data is required.

```
y[k] = even[k] + w * odd[k]
```

- **Structured control flow:** Iterative computation must be expressed clearly.

```
while k < n/2 {  
    ...  
}
```

These features are typically provided implicitly by modern programming languages. This project investigates whether they can be reconstructed within a minimal execution system.

Primary research question:

Can the Fast Fourier Transform be implemented correctly within a minimal computational system while preserving its theoretical complexity?

Secondary research question:

Can a minimal programming language express and execute an optimal $O(n \log n)$ algorithm such as FFT without relying on external abstractions?

III. SYSTEM ARCHITECTURE

The execution of a program in Seralix follows a multi-stage pipeline. Each stage transforms the program into a more structured representation until it can be executed.

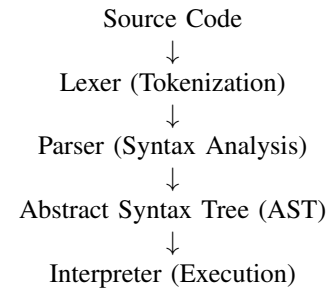


Fig. 1. Execution pipeline of Seralix

A. Lexer (Tokenization)

The lexer converts raw text into tokens.

Example:

```
x = 10
```

becomes:

```
[NAME(x), '=', NUMBER(10)]
```

This step removes whitespace and groups characters into meaningful units.

B. Parser (Syntax Analysis)

The parser organizes tokens into a structured representation based on grammar rules.

Example:

```
x = 10
```

becomes:

```
Assignment(  
  target=Name('x'),  
  value=Number(10)  
)
```

This structure defines how expressions are interpreted.

C. Abstract Syntax Tree (AST)

The AST is a tree representation of the program where each node corresponds to an operation.

Example:

```
x * y + 2
```

becomes:

```
BinaryOp('+',  
  left=BinaryOp('*', x, y),  
  right=2  
)
```

This representation enforces operator precedence and evaluation order.

D. Interpreter (Execution)

The interpreter evaluates the AST node by node.

Example:

```
x = 10  
y = 5  
x * y
```

Execution:

- Assign $x = 10$
- Assign $y = 5$
- Compute $x * y = 50$

This stage produces the final result of the program.

IV. FUNDAMENTAL THEORY

A. Discrete Fourier Transform

The DFT of a sequence $a_0, \dots, a_{n-1}$ is:

$$A_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk} \quad (1)$$

where $\omega_n = e^{2\pi i/n}$.

The direct computation requires:

$$O(n^2) \quad (2)$$

B. FFT Decomposition

FFT reduces this complexity by splitting the sequence into even and odd indices:

$$A_k = A_k^{(even)} + \omega_n^k A_k^{(odd)} \quad (3)$$

This produces the recurrence:

$$T(n) = 2T(n/2) + O(n) \quad (4)$$

which resolves to:

$$O(n \log n) \quad (5)$$

V. LANGUAGE DESIGN

Seralix combines functional and imperative constructs to support algorithm implementation.

A. Functions

```
(x) -> x * x
```

```
function add(x, y=2) {  
  return x + y;  
}
```

B. Control Flow

```
if condition {  
  ...  
} else {  
  ...  
}
```

C. Loops

```
while i < 10 {  
  i += 1;  
}
```

VI. FFT IMPLEMENTATION

```
function fft(a, invert=false) {  
  if len(a) == 1 {  
    return [a[0]];  
  }  
  
  even = fft(a[0::2], invert);  
  odd = fft(a[1::2], invert);  
}
```

VII. RESULTS

```
a = [1, 2, 3];  
b = [4, 5, 6];
```

Output:

```
[4, 13, 28, 27, 18]
```

VIII. DISCUSSION

The implementation confirms that FFT can be expressed within a minimal computational system. However, several limitations remain:

- Lack of optimized memory access patterns
- Floating-point precision limitations
- Missing low-level numerical optimizations

IX. CONCLUSION

This project demonstrates that it is possible to implement FFT correctly within a minimal computational framework. The results validate both the correctness of the algorithm and the feasibility of constructing such a system from first principles.

X. FUTURE WORK

Future improvements should focus on:

- Iterative (non-recursive) FFT implementations for performance
- Memory layout optimization to improve cache efficiency
- Integration of exact arithmetic approaches such as NTT
- Introduction of a static type system to improve numerical safety and detect errors at compile time